

UCL Erhvervsakademi og Professionshøjskole
Datamatikeruddannelsen

DMVE251 - 2. Semester Tværfaglige Prøve

Eksamensopgave

Prøve	2. semester tværfaglig prøve
ECTS	35 ECTS
Type	Ekstern prøve, 7-trinsskala
Varighed	4 uger
Gruppetørrelse	3–6 studerende
Teknologi	C# / .NET 10 / Blazor
Arkitektur	Clean Architecture med Facade

Semester: Forår 2026

1. Introduktion

Denne eksamensopgave udgør den tværfaglige prøve efter 2. semester på Datamatikeruddannelsen. Opgaven dækker alle fire nationale fagelementer på 2. semester og skal løses i grupper af 3–6 studerende over en periode på 4 uger.

Prøven er ekstern med bedømmelse efter 7-trinsskalaen. Der gives én samlet individuel karakter ud fra en helhedsvurdering af den skriftlige rapport og den mundtlige eksamination.

2. Fagelementer og ECTS

Opgaven integrerer følgende nationale fagelementer:

Fagelement	ECTS	Semester
Programmering I	15	2.
Systemudvikling I	7,5	2.
Teknologi I	2,5	2.
IT- og Forretningsudvikling	5	2.
I alt	35 ECTS	

3. Opgavebeskrivelse

3.1 Casebeskrivelse

BookRight Klinik & Wellness ApS er en fiktiv dansk virksomhed, der driver tre klinikker i Vejle-området. Virksomheden tilbyder fysioterapi, massage, akupunktur og kostvejledning. Klinikkerne har tilsammen 12 behandlere med forskellige specialer og autorisationer.

BookRight har i dag en manuel proces baseret på en kombination af telefonisk booking, et fælles regneark og håndskrevne blokke i receptionen. Systemet fører regelmæssigt til dobbeltbookinger, tabte aftaler og manglende overblik over behandlernes belægning.

Klinikkerne deler behandlere, så en fysioterapeut kan arbejde i Vejle om formiddagen og i Egtved om eftermiddagen. Denne fleksibilitet gør planlægningen ekstra kompleks.

Systemet er et **internt administrationssystem** til brug for receptionisterne. Der er kun én brugerrolle: **receptionist**. Receptionisten opretter og administrerer bookinger, registrerer kunder, håndterer afbud og betjener kunder ved ankomst.

BookRight ønsker et nyt IT-system, der kan:

- Håndtere booking af behandlinger på tværs af klinikker og behandlere
- Sikre mod dobbeltbookinger
- Beregne den for kunden bedste pris baseret på loyalitetsniveau, kampagner og fødselsdagsmåned.
- Visualisere behandlernes kalender og klinikernes belægning via et visuelt dashboard.
- Generere omsætningsrapporter, der kan deles med ledelsen.

Forretningskontekst: BookRight har 3 klinikker, 12 behandlere og betjener ca. 400 kunder månedligt. Virksomheden forventer at åbne en fjerde klinik inden for det næste år. Den nuværende manuelle løsning koster anslået 15 timer ugentligt i administration og har ført til et estimeret tab på 8–10 % af potentielle bookinger pga. dobbeltbookinger og manglende opfølgning.

Behandlingstyper og prissætning

BookRight tilbyder følgende behandlingstyper:

- Fysioterapi (30, 45 eller 60 min.) – 395 kr, 589 kr, 745 kr.
- Sportsmassage (30 eller 60 min.) – 350 kr, 699 kr.
- Akupunktur (45 min.) – 550 kr.
- Kostvejledning (60 min. førstegangskonsultation, 30 min. opfølgning) – førstegang 799kr, og efterfølgende 450 kr.
- Holdtræning/genoptræning (60 min., max 6 deltagere) – 150 kr. pr. deltager

Priserne afhænger af behandlerens autorisationsniveau. Autoriserede fysioterapeuter har en højere timepris end massører. Ved aften- og weekendbookinger tillægges 15%. Nogle behandlinger kan kombineres i samme besøg (f.eks. fysioterapi + massage), hvilket kræver sammenhængende tidsblokke.

Rabat- og prissystem

BookRight ønsker et rabatsystem, der automatisk beregner **den for kunden bedste pris** ved oprettelse af en booking. Receptionisten skal ved booking kunne se kundens pris med og uden rabat samt hvilken rabattype, der er anvendt.

Følgende rabattyper skal understøttes:

Rabattype	Betingelse	Rabat
Bronze-loyalitet	Køb for 3.000–10.000 kr. (seneste 12 mdr.)	5% på alle behandlinger
Sølv-loyalitet	Køb for 10.001–25.000 kr. (seneste 12 mdr.)	10% på alle behandlinger

Guld-loyalitet	Køb for over 25.000 kr. (seneste 12 mdr.)	15% på alle behandlinger
Fødselsdagsmåned	Booking i kundens fødselsmåned	25% på én behandling
Kampagnerabat	Tidsbegrænsede kampagner (sæson, tema)	Varierende (f.eks. 20%)

Loyalitetsniveauer beregnes ud fra kundens samlede køb (afsluttede bookinger) de seneste 12 måneder. Niveauet evalueres dynamisk ved hver booking.

Fødselsdagsmånedsrabat gælder for bookinger, der ligger i kundens fødselsmåned. Rabatten gælder kun for én behandling pr. fødselsmåned.

Kampagnerabatter er tidsbegrænsede tilbud, som administrationen opretter (f.eks. "Forårskampagne: 20% på massage i april"). En kampagne har en start- og slutdato samt en liste af behandlingstyper, den gælder for.

Når flere rabatter er relevante, vælges automatisk **den rabat, der giver kunden den laveste pris**. Rabatterne kombineres ikke – kun den bedste enkeltstående rabat anvendes.

Klinikker og behandlere

Om de tre klinikker registreres: adresse, åbningstider (kan variere pr. ugedag) og antal behandlingsrum. Hver klinik har et begrænset antal rum, hvilket sætter en øvre grænse for samtidige bookinger.

Om behandlerne registreres: navn, kontaktoplysninger, autorisationstype (fysioterapeut, massør, akupunktør, kostvejleder), autorisationsnummer og de klinikker, behandleren er tilknyttet. Ikke alle behandlere kan udføre alle behandlingstyper – dette styres af deres autorisationer. En behandler er til stede på en klinik en hel dag af gangen.

Kunder og booking-historik

Om kunderne registreres: fornavn, efternavn, telefon, e-mail, fødselsdato, adresse, eventuelle helbredsnotater (allergier, skader o.l.) og en reference til foretrukken behandler.

Systemet skal holde historik over alle bookinger for en given kunde, herunder afsluttede, aflyste og no-shows. Denne historik danner grundlag for loyalitetsniveau-beregning, prisberegning og belægningsstatistik.

3.2 Opgave

I skal som gruppe analysere, designe og implementere en prototype af BookRight-systemet i C# / .NET 10.

Som grundprincip skal I anvende OOP-principperne, hvor det er meningsfuldt.

Prototypen skal desuden bygges med:

- Blazor som brugergrænseflade
- Clean Architecture som arkitekturmønster
- SOLID-principperne
- Automatiserede tests

Nedenfor beskrives kravene, opdelt efter fagelement.

3.2.1 Programmering I

A) Prototype med SOLID, Clean Architecture og Blazor

- Strukturer løsningen i lagene Domain, Use Case, Facade, Infrastructure og UI (Blazor) som separate .NET-projekter i én solution.
- Implementer DDD aggregate roots, entity-klasser og value objects.
- Anvend CQS (Command Query Separation).
- Implementer Facade-laget med interfaces og DTO'er (C# records) som eneste kontrakt mod Blazor.
- Byg brugergrænsefladen i Blazor med komponenter, der injicerer Facade-interfaces via dependency injection. Systemet har én rolle: receptionist.
- Anvend Entity Framework Core Code First med mest mulig implicit konfiguration. Brug kun eksplicit konfiguration (Fluent API / Data Annotations), når konventionerne ikke rækker.
- Anvend designmønstre. Som minimum: Repository, Dependency Injection og Strategy (til rabatberegning).
- Anvend SOLID-principperne.

B) Rabatberegning med Strategy Pattern

- Implementer rabatberegning med Strategy Pattern.
- Når en booking oprettes, samles alle relevante *IRabatBeregner*-strategier og afvikles parallelt (CPU-bound). Hver strategi skriver sit resultat til et fælles resultatobjekt, og den rabat, der giver kunden den laveste pris, vælges automatisk.

C) Parallelisme – I/O-bound og CPU-bound

- **I/O-bound:** Anvend async/await, hvor det er muligt (f.eks. ved databasekald).
- **CPU-bound:** Anvend CPU-bound tasks til beregningsintensive opgaver (f.eks. parallel rabatberegning og evt. parallel generering af belægningsstatistik pr. behandler/klinik til dashboard).
- Dokumentér og argumentér i rapporten for, hvor og hvorfor I anvender de forskellige paralleliseringsteknikker.

D) Test og kvalitetssikring

- Skriv unit tests for Domain-laget.
- Skriv unit tests for Use Cases med Moq.

- Anvend Git med et struktureret workflow (branches og commits med meningsfulde beskeder).

3.2.2 Systemudvikling I

I skal gennemføre en systematisk udviklingsproces og dokumentere denne:

- Anvend Scrum som systemudviklingsmetode.
- Udarbejd kravspecifikation med funktionelle og ikke-funktionelle krav.
- Modeller systemet med UML: som minimum use case-diagram, klassediagram og mindst et sekvensdiagram for en command og mindst et for en query (sekvensdiagrammet skal vise flowet gennem lagene – f.eks. en OpretBooking-kommando fra Blazor gennem Facade, Use Case, Domain Service og ned til Infrastructure)
- **C4-modellen:** Anvend C4-modellen til dokumentation af systemets arkitektur. Som minimum skal der udarbejdes et Context-diagram (C1) og et Container-diagram (C2). Component-diagrammer (C3) for udvalgte containere er valgfrit, men anbefales for at vise Clean Architecture-lagdelingen.
- **Architecture Decision Records (ADR):** Dokumenter centrale arkitektoniske beslutninger ved hjælp af Architecture Decision Records. Hver ADR skal som minimum indeholde: titel, kontekst, beslutning og konsekvenser.
- Planlæg og udfør test og kvalitetssikring med dokumentation af teststrategi og testresultater.
- Demonstrer sporbarhed fra krav til design til implementering.

3.2.3 Teknologi I

I skal demonstrere forståelse for de teknologiske lag i systemet:

- **Code First og normalisering:** Databasen oprettes via EF Core Code First ud fra jeres domænemodel. I rapporten skal I analysere og vurdere den resulterende database i forhold til normaliseringsformerne (1NF–3NF). Formålet er ikke at designe databasen separat, men at demonstrere forståelse for normalisering ved at evaluere det skema, EF Core genererer.
- Redegørelse for relevante operativsystem-begreber i relation til systemets drift (tråde, processer, I/O)
- Redegør for sammenhængen mellem I/O-bound vs. CPU-bound parallelisme og operativsystemets håndtering af tråde

3.2.4 IT- og Forretningsudvikling

I skal sætte systemudviklingen i en forretningsmæssig kontekst:

- Analyse af BookRights forretningsprocesser og identifikation af, hvordan IT-systemet forbedrer disse
- Vurdering af organisatoriske forandringer ved implementering af systemet (f.eks. fra telefonbooking til digital booking, ændrede arbejdsgange for receptionisterne)
- Overvejelser om IT-Governance og bæredygtigt IT i relation til projektet
- Formidling af projektets status – simuler en statusrapport til BookRights ledelse
- Reflektér over anvendelsen af innovative metoder i projektarbejdet

4. Afleveringskrav

4.1 Projektrapport

Der skal afleveres en skriftlig projektrapport, som dokumenterer hele udviklingsprocessen.

Formkrav:

Gruppestørrelse	Anslag inkl. mellemrum
3 studerende	24.000–72.000
4 studerende	24.000–84.000
5-6 studerende	24.000–96.000

- En figur tæller for 400 anslag.
- Forside, indholdsfortegnelse, litteraturliste og bilag tæller ikke med i antal anslag.
- Bilag er uden for bedømmelse.

Rapportens forventede indhold:

- Indledning, der præsenterer udfordringen, virksomheden står med
- Evt. problemformulering og evt. afgrænsninger.
- Metodeafsnit med jeres brug af Scrum og andre elementer, der beskriver jeres udviklingsproces.
- Forretningsanalyse (forretningsprocesser før og efter, evt. mere der er relevant for casen)
- Forretningsanalyse (forretningsprocesser før og efter, evt. mere der er relevant for casen).
- Kravspecifikation (funktionelle og ikke-funktionelle krav).
- Arkitektur og systemdesign (Clean Architecture-lagdeling, Dependency Rule, CQS, C4-modellen, Architecture Decision Records (ADR), UML-modellering (vælge relevante diagrammer)).
- Sprint der dokumenterer centrale handlinger og beslutninger, så vi får et indblik i jeres proces og produkt fra sprint til sprint.
- Konklusion og evt. refleksionsafsnit.

4.2 Sourcekode og Git-repository

Al sourcekode skal afleveres via et Git-repository. Repositoryet skal:

- Indeholde en README med instruktioner til opsætning og kørsel af systemet
- Følge Clean Architecture-projektstrukturen med separate projekter pr. lag
- Have en struktureret commit-historik, der viser gruppens udviklingsproces
- Demonstrere brug af branches og/eller pull requests
- Indeholde relevante konfigurationsfiler (men ikke følsomme data som passwords eller connection strings)

Link til Git-repositoryet skal fremgå af rapportens forside.

5. Mundtlig prøve

Den mundtlige prøve består af to dele:

Gruppepræsentation (30 minutter – inkl. ca. 5 minutters votering):

Gruppen præsenterer projektet samlet. Præsentationen skal som minimum indeholde en demonstration af det kørende system.

Individuel eksamination (30 minutter pr. studerende inkl. votering):

Herefter foregår individuel eksamination med udgangspunkt i projektarbejdet samt læringsmålene for alle fire fagelementer.

6. Praktiske oplysninger

	Detaljer
Projektperiode	4 uger
Gruppestørrelse	3–6 studerende (dispensation kan søges)
Teknologi	C# / .NET 10 / Blazor / Entity Framework Core
Arkitektur	Clean Architecture med DDD, CQS og Facade-lag
Aflevering	Projektrapport uploades i WISEflow. Link til Git-repository angives på forsiden.
Mundtlig prøve	30 min. gruppepræsentation + 30 min. individuel eksamination inkl. votering
Bedømmelse	Ekstern prøve. Individuel karakter efter 7-trinsskalaen.

7. HUSK!!!!

- Spørg, hvis I er i tvivl om, hvordan opgaven skal fortolkes.
- Brug muligheden for vejledning i projektperioden.

God arbejdslyst!